

Documentazione Tecnica

Sistema VLM + YOLO + AI2-THOR Allineamento completo al codice corrente Data: 2026-02-06

1. Scopo e Ambito

Questo documento descrive il comportamento reale dell'architettura in Richiesta 1/src/Architettura:

- avvio simulatore AI2-THOR;
- ciclo decisionale VLM con validazione e retry;
- produzione artefatti visivi e output VLM;
- esecuzione azioni AI2-THOR da payload JSON;
- generazione documentazione automatica (overlay + video);
- raccolta metriche e benchmark summary.

Obiettivo operativo: rendere tracciabile il flusso end-to-end senza assunzioni obsolete.

2. Architettura dei Moduli

Moduli principali:

- Architettura/app/start.py: orchestratore runtime (init, loop, VLM, action execution, summary, documentazione finale).
- Architettura/ai2thor/init_controller.py: configurazione SimConfig e creazione controller.
- Architettura/ai2thor/utils.py: salvataggio artefatti first/third-person + integrazione YOLO.
- Architettura/ai2thor/context.py: costruzione contesto VLM da metadata/evento.
- Architettura/vlm/client.py: factory backend HF/GGUF e gestione prompt/generazione.
- Architettura/vlm/vlm_execute_llama.py: backend GGUF con llama.cpp.
- Architettura/vlm/runner.py: prepara prompt+immagini, invoca VLM, salva diagnostica/risposte.
- Architettura/vlm/agent/action_executor.py: parse JSON robusto, validazione semantica, esecuzione azioni.
- Architettura/yolo/utils.py: overlay YOLO (save_yolo_png) e metadati target nel frame.
- Architettura/Documentazione_e_Test/run/start_documentation.py: overlay/video + pulizia artefatti.
- Architettura/Documentazione_e_Test/run/benchmark_models.py: benchmark multi-modello con report.

3. Avvio e Configurazione Runtime

Entrypoint: Architettura/app/start.py.

Configurazione attuale:

- scena default: FloorPlan1;
- risoluzione controller: 1920x1080;
- renderDepthImage=True;

- `renderInstanceSegmentation=True`;
- `renderSemanticSegmentation=True`;
- third-person camera aggiunta con `AddThirdPartyCamera`.

Sequenza di avvio:

- Lettura goal da CLI (`sys.argv[1]`, opzionale).
- Lettura env vars runtime (modello/preset/soglie).
- Creazione controller con `create_controller`.
- Setup camera terza persona.
- Setup VLM (backend HF o GGUF) + preload modello.
- Primo Pass per ottenere evento iniziale.

4. Variabili Ambiente Effettive

Variabili usate nel runtime principale:

- `VLM_PROMPT`: seleziona il prompt di sistema (`vlm_prompt.py`).
- `VLM_MODEL`: alias o model id (default `qwen-vl2.5`).
- `VLM_MM PROJ`: alias/path `mmproj` per GGUF (default `qwen-vl-mmproj-2.5`).
- `VLM_PRESET`: preset di generazione (accurate o fast, default accurate).
- `VLM_MAX_STEPS`: massimo step del loop (default 170).
- `VLM_VALIDATION_RETRIES`: retry massimi per JSON/semantica (default 2).
- `VLM_FINAL_DISTANCE_THRESHOLD`: soglia metrica finale (default 0.8).
- `VLM_FINAL_DISTANCE_TOLERANCE`: tolleranza aggiuntiva (default 0.03).
- `YOLO_DEVICE`: se backend GGUF e variabile assente, viene forzata a cpu.
- `YOLO_TARGET_CONF`: confidenza minima usata in `save_yolo_png` (default 0.40).
- `BENCHMARK_SUMMARY_PATH`: path JSON per summary a fine run.
- `DOC_SIM_NAME`: nome simulazione per output documentazione (`SIM_n` se assente).

Variabile impostata internamente:

- `PYTORCH_ALLOC_CONF=expandable_segments=True`.

5. Ciclo Decisionale Reale

Per ogni step:

- Aggiorna third-person camera.
- Salva artefatti first/third-person.
- Esegue YOLO overlay (`save_yolo_png`) e raccoglie:
 - `yolo_target_match`,
 - `yolo_track_id` (attualmente None),
 - `yolo_target_center_x`.

- Calcola `target_distance` dal metadata AI2-THOR.
- Invoca `run_vlm_on_step`.
- Tenta parse+validazione payload VLM con retry.
- Esegue azioni tramite `execute_vlm_payload`.
- Gestisce stop/fail e aggiorna stato locale (`last_state`, `scan_count`, `telemetry`).

Retry e fallback

Per ogni step è attivo un ciclo di tentativi:

- parse JSON robusto (`parse_vlm_json`);
- validazione semantica (`validate_vlm_semantics`);
- in caso errore: prompt di correzione con blocco `VALIDATION_ERROR`;
- a retry esauriti: Pass con errore logico.

Metriche incrementate:

- `json_parse_failures_count`;
- `semantic_validation_failures_count`;
- `retry_attempts_total`;
- `action_failures_count`.

6. Contesto VLM e Immagini

`build_context` produce dati base da metadata/evento, ma `run_vlm_on_step` applica pulizia/arricchimento prima della chiamata VLM.

Campi effettivi nel contesto finale inviato al prompt:

- `last_action`;
- `last_action_success`;
- `last_action_error`;
- `collided`;
- `collidedObjects`;
- `target_object`;
- `target_distance`;
- `yolo_target_match` (aggiunto nel runner);
- `last_state` (aggiunto nel runner);
- `target_center_x` (aggiunto se disponibile).

Note importanti:

- `scan_count` viene rimosso dal context finale nel runner;
- `last_seen` viene rimosso dal context finale nel runner;
- `current_state` viene estratto dalla risposta VLM e salvato come diagnostica.

Input immagini VLM:

- preferenza semantic frame;
- fallback RGB frame;
- seconda immagine opzionale: `oracle_overlay` oppure `yolo` (attualmente usato `yolo`).

7. Pipeline YOLO/Overlay (Stato Attuale)

Comportamento operativo:

- `save_artifacts_yolo(...)` è minimale e ritorna solo `{"yolo_seg": None}`;
- overlay utile viene generato da `save_yolo_png(...)`;
- `yolo_target_match` deriva da match YOLO, con fallback `metadata_target_visible` da AI2-THOR.

Conseguenza pratica:

- il flusso non usa un JSON YOLO ricco come fonte primaria nel loop;
- eventuali riferimenti a pipeline segmentazione completa devono essere considerati opzionali/non attivi nel path principale.

8. Action Executor: Parse, Sicurezza e Vincoli

`action_executor.py` implementa:

- whitelist azioni consentite:

`MoveAhead`, `MoveBack`, `MoveLeft`, `MoveRight`, `RotateRight`, `RotateLeft`, `LookUp`, `LookDown`, `Crouch`, `Stand`, `Stop`;

- parsing robusto;
- rimozione code fences,
- estrazione JSON bilanciato,
- tentativo repair,
- fallback da testo libero;
- normalizzazione parametri (`moveMagnitudeCm` -> `moveMagnitude`, `magnitude` -> `moveMagnitude`);
- supporto `action_sequence` oltre a singola `action`.

Validazione semantica:

- se distanza target è sopra soglia finale, vengono bloccate proposte di stato/azioni finali (FINE, Stop);
- gli errori semantici alimentano il ciclo di retry nel runtime.

9. Telemetria e Benchmark Summary

A fine run, `start.py` compone un summary JSON con:

- metadati run (`run_started_at`, `run_ended_at`, `runtime_sec`);
- esito (`exit_reason`, `success`, `target_reached`);
- metrica distanza (`min_target_distance_m`, `final_target_distance_m`, `time_to_target_sec`);

- qualità decisionale (json_parse_failures_count, semantic_validation_failures_count, retry_attempts_total);
- performance (vlm_inference_mean_sec, vlm_inference_p95_sec, model_load_sec);
- configurazione finale (final_distance_threshold, final_distance_tolerance, final_distance_limit).

Se BENCHMARK_SUMMARY_PATH è valorizzato, il JSON viene scritto su file.

10. Documentazione Automatica (Overlay + Video)

La documentazione viene avviata nel blocco finally di start.py:

- chiamata a run_documentation(sim_name=DOC_SIM_NAME|auto);
- se sim_name assente, start_documentation.py crea SIM_n progressivo;
- genera overlay e video nella cartella:

src/Artefatti/Documentazione/SIM_n/;

- esegue pulizia artefatti temporanei con clear_artifacts(ARTIFACTS_ROOT).

Gestione interruzione:

- su KeyboardInterrupt il runtime non perde la fase di chiusura/documentazione;
- in benchmark mode viene evitato prompt interattivo.

11. Pseudocodice End-to-End Aggiornato

```
goal = CLI_ARG_OR_EMPTY
read env (VLM_MODEL, VLMPROMPT, preset, retries, thresholds, ...)

controller = create_controller(scene="FloorPlan1")
add_third_party_camera()
vlm = create_vlm(...)
vlm.load()

event = controller.step("Pass")
init scan_count, scan_complete, telemetry, last_state

for step_idx in range(max_steps):
    update_third_person_camera(controller, event)
    paths = save_artifacts(event, step_idx, target_label=current_target)

    target_distance = min_distance_from_metadata(event.metadata.objects, current_target)
    telemetry.update_distance(target_distance)

    base_prompt = build_step_prompt(...)
    vlm_output = run_vlm_on_step(..., paths=paths, last_state=last_state)

    result = None
    retry_prompt = base_prompt
    for attempt in range(validation_retries + 1):
        payload = parse_vlm_json(vlm_output) or fallback
        semantic_errors = validate_vlm_semantics(payload, target_distance, threshold, tolerance)
        if parse_or_semantic_error and attempt < validation_retries:
            retry_prompt = add_VALIDATION_ERROR(base_prompt, details)
            vlm_output = run_vlm_on_step(..., user_prompt=retry_prompt)
            continue
        result = execute_vlm_payload(controller, payload) or Pass_on_failure
    break
```

```

    if result.action == "Stop":
        save_final_frame_for_overlay_shift()
        break

    event = result.event
    update_scan_count/scan_complete based on executed actions
    telemetry.update_failures(result)

finally:
    run_documentation(sim_name=DOC_SIM_NAME or auto_SIM_n)
    controller.stop()
    compute summary metrics
    if BENCHMARK_SUMMARY_PATH: write summary JSON

```

12. Output Artefatti (Path Reali)

Root artefatti:

- src/Artefatti/.

Sottocartelle principali:

- src/Artefatti/vision_outputs/first_person/rgb/;
- src/Artefatti/vision_outputs/first_person/depth/;
- src/Artefatti/vision_outputs/first_person/instance/;
- src/Artefatti/vision_outputs/first_person/semantic/;
- src/Artefatti/vision_outputs/third_person/;
- src/Artefatti/yolo_outputs/yolo/;
- src/Artefatti/vlm_outputs/responses/;
- src/Artefatti/diagnostic/;
- src/Artefatti/Documentazione/SIM_n/{overlays,videos}/.

13. Build e Versionamento della Documentazione

Sorgente primaria:

- docs/Documentazione_Sistema_VLM_YOLO_AI2THOR.md.

Generazione PDF:

```
python3 docs/build_system_doc_pdf.py
```

Output default:

- docs/Documentazione_Sistema_VLM_YOLO_AI2THOR_YYYYMMDD.pdf.

Build con path espliciti:

```

python3 docs/build_system_doc_pdf.py \
  --in docs/Documentazione_Sistema_VLM_YOLO_AI2THOR.md \
  --out docs/Documentazione_Sistema_VLM_YOLO_AI2THOR_20260206.pdf

```

14. Known Limits

- Generatore PDF basato su ReportLab (python3 di sistema): richiede libreria disponibile nell'ambiente.
- Parser Markdown intentionally minimale: supporta bene heading/paragrafi/code blocks; feature avanzate Markdown non sono target.

- Alcuni campi YOLO avanzati (tracking persistente/JSON ricco) non sono nel path principale corrente.
- Le prestazioni dipendono da backend VLM (HF vs GGUF), preset e disponibilit  GPU.